

Day 27: Automated Matrix Inversion Programming

NUMPY RELEASES, 4X4 BUSINESS AUTOMATION ENGINE & 1-LINE CODE INTERPRETATION

1. Introduction to Programmatic Inversion

English: Manual matrix inversion is highly prone to human arithmetic errors as dimensions scale. Python's NumPy library handles these backend linear algebra transformations seamlessly via compiled C-extensions, ensuring maximum precision and multi-thread performance.

Haath se matrix inversion karne par dimensions badhne ke sath human mathematical errors badhne ka khata rehta hai. Python ki NumPy library in linear algebra equations ko back-end par bina kisi error ke behtareen precision aur speed ke sath processing karti hai.

2. Comprehensive Code Script (Detailed Production Script)

English: Below is the complete production script mapping the 4x4 Goda Gold showroom transactional scenario. It automatically computes the determinant, the exact inverse matrix array, and extracts final segment weights instantly.

Neeche poora complete production script hai jo 4x4 Goda Gold showroom ka business profile map karta hai. Yeh script determinant, accurate inverse array aur final customer answers ek baar mein generate karta hai.

```

import numpy as np

# Coefficients Matrix (A) based on metrics
A = np.array([
    [2, 10, 25, 5],
    [100, 500, 2000, 300],
    [1, 4, 10, 2],
    [15, 60, 120, 30]
])

# Target Results Vector (B)
B = np.array([160, 9900, 66, 930])

print("=====GODA GOLD AUTOMATED ACCOUNTING ENGINE====")

# Compute Determinant and Inverse Layout
det_A = np.linalg.det(A)
A_inverse = np.linalg.inv(A)

# Calculate exact results vector (X = A_inv * B)
X_solutions = np.linalg.solve(A, B)

print(f"
1. DETERMINANT (|A|): {round(det_A, 2)}")
print("
2. INVERSE MATRIX (A^-1):")
print(np.round(A_inverse, 4))

print("
3. FINAL SEGMENT ANALYSIS SOLUTIONS (X):")
customer_types = ["Retail (x1)", "Corporate (x2)", "VIP (x3)", "Tourists (x4)"]
for i, customer in enumerate(customer_types):
    print(f" -> {customer} = {round(X_solutions[i])}")
print("=====")

```

3. Hyper-Optimized Ultra Short Solution (1-Line Code Engine)

English: For instant deployment inside production data analysis pipelines, the entire process can be condensed into a single high-efficiency line using `np.linalg.solve()` to instantly yield raw matrix values without staging separate computational layers.

Production algorithms mein jaldi deployment ke liye, pure complex calculation ko sirf ek line mein convert kiya ja sakta hai jahan `np.linalg.solve()` seedhe answers nikaal deta hai.

```
import numpy as np

A = np.array([[2,10,25,5], [100,500,2000,300], [1,4,10,2], [15,60,120,30]])
B = np.array([160, 9900, 66, 930])

# SEEDHE EK LINE ME ANSWER EXTRACATION:
print("Final Answers (x1, x2, x3, x4) ->", np.round(np.linalg.solve(A, B)))
```

Technical Core Breakdown

English: The `np.linalg.solve(A, B)` execution routine does not simply calculate the literal algebraic inverse. Instead, it applies advanced LAPACK factorizations (like LU decomposition) which avoids mathematical instability and handles float processing faster than any explicit manual transposition loops.

`np.linalg.solve(A, B)` sirf literal algebraic inverse nahi nikalta, balki back-end par heavy factorizations use karta hai jo memory blocks ko efficient rakhte hain aur decimals rounding errors ko bilkul khatam kar dete hain.